

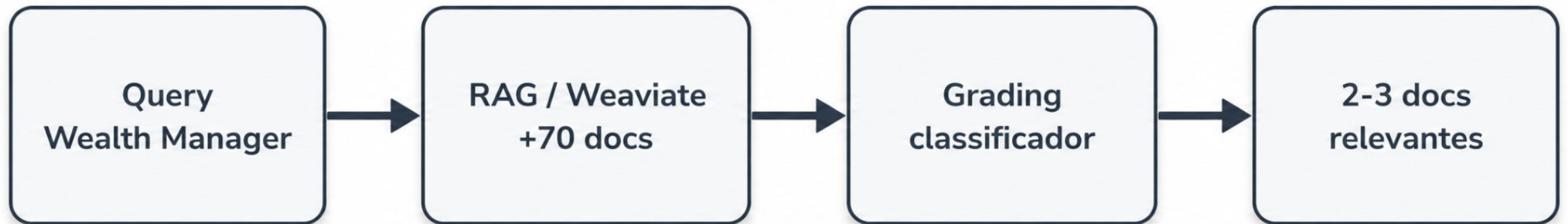
# Add-ons para Regressão

## Logística em Class

## Imbalance

Bruno Souza Zabet

# Motivação

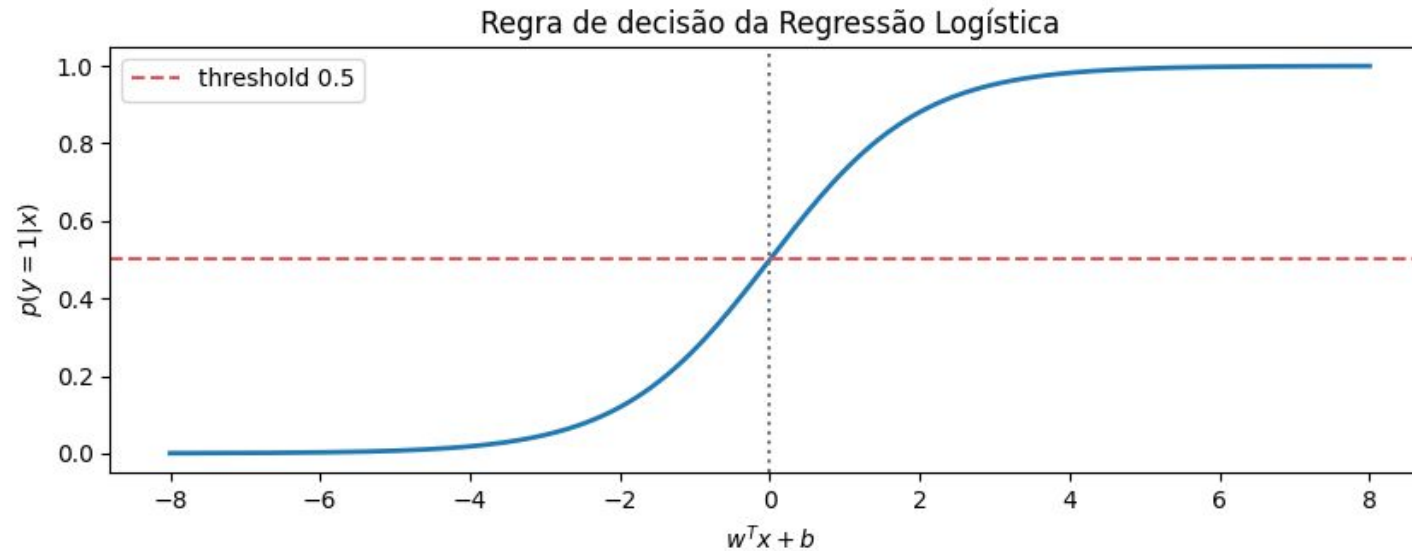


# Sumário

| Modelo                   | Balanced Accuracy | Minority Recall | Minority F1 Score |
|--------------------------|-------------------|-----------------|-------------------|
| CAGD + Dynamic Threshold | 0.7907            | 0.7056          | 0.6048            |
| GAAR + CAGD              | 0.7869            | 0.6106          | 0.5614            |
| Default                  | 0.6639            | 0.3352          | 0.3722            |

- Objetivo: melhorar a detecção da classe minoritária sem destruir a classe majoritária.
- Baseline: Regressão Logística default, loss simétrica e threshold 0.5.
- Proposta: 4 add-ons combináveis.
- Estudo: 19 datasets x 20 seeds x 16 modelos = 6080 treinos.

# Regressão Logística e Efeito do Desequilíbrio



- Classe positiva é rara, mas em muitos problemas é a classe de interesse (documentos RAG)
- Um modelo que prevê a maioria sempre tem a accuracy altíssima, mas não é usável
- Desafio não é só acertar muitos exemplos, mas reconhecer a minoria sem gerar ruído

# Métricas

|      |                         | Predito                                      |                                                  |
|------|-------------------------|----------------------------------------------|--------------------------------------------------|
|      |                         | Positivo                                     | Negativo                                         |
| Real | Positivo<br>100 casos   | <b>TP</b><br><b>15</b><br>15% dos positivos  | <b>FN</b><br><b>85</b><br>85% dos positivos      |
|      | Negativo<br>9.900 casos | <b>FP</b><br><b>50</b><br>0,5% dos negativos | <b>TN</b><br><b>9.850</b><br>99,5% dos negativos |

10.000 observações: 100 positivos reais (1%) e 9.900 negativos reais (99%)

Acurácia = 98,65%; recall da classe positiva = 15%

**Majority recall:** percentagem dos exemplos da classe 0 reconhecidos como 0

**Minority recall:** percentagem dos exemplos da classe 1 reconhecidos como 1

**Balanced accuracy:** média dos dois recalls

**Minority precision:** entre as previsões positivas, quantas estavam certas

**Minority F1:** compromisso entre precision e recall da classe minoritária

# Add-ons - Weighted Errors (Focal Loss)

- Proposto por Lin et al (2017), Focal Loss for Dense Object Detection
- Quanto maior a confiança correta do modelo, menor o peso daquele exemplo no erro
- Força o modelo a prestar mais atenção a casos difíceis, logo pode ajudar a melhorar o class imbalance

| y | p   | Erro |
|---|-----|------|
| 1 | 0.9 | 0.1  |
| 1 | 0.2 | 0.8  |

8x

$$\text{Peso} = \alpha * (1-p_t)^\gamma$$

| y | p   | Erro | Peso  | Peso * Erro |
|---|-----|------|-------|-------------|
| 1 | 0.9 | 0.1  | 0.005 | 0.0005      |
| 1 | 0.2 | 0.8  | 0.320 | 0.2560      |

512x

# Add-ons - Gradient Asymmetric Additive Regularization

— Adiciona uma penalidade (OMEGA) no coeficiente  $d_w$  do gradient descent

$$\Omega = \frac{\lambda}{n} \sum_{i=1}^n c_i ||\nabla_w l_i(w)||^2$$

Para Regressão Logística,  $\nabla_w l_i(w) = (p_i - y_i)x_i$ . Logo:

$$\nabla_w \Omega = \frac{\lambda}{n} \sum_{i=1}^n 2c_i(p_i - y_i)p_i(1 - p_i)||x_i||^2 x_i$$

# Add-ons - Gradient Asymmetric Additive Regularization

$$\Omega = \frac{\lambda}{n} \sum_{i=1}^n c_i ||\nabla_w l_i(w)||^2$$

Para Regressão Logística,  $\nabla_w l_i(w) = (p_i - y_i)x_i$ . Logo:

$$\nabla_w \Omega = \frac{\lambda}{n} \sum_{i=1}^n 2c_i(p_i - y_i)p_i(1 - p_i)||x_i||^2 x_i$$

Exemplo com  $\lambda=0.1$  e  $c_{min}=10$ :

| X | y | erro | pesos iniciais (w) |
|---|---|------|--------------------|
| 2 | 1 | -0,5 | 0                  |
| 1 | 0 | 0,5  | 0                  |

Com pesos iniciais a zero, o sigmoide é 0.5 para as duas amostras, logo  $P=[0.50,0.50]$

$$d_w = np.dot(X.T, error)/len(y)$$

$$d_w = ([2, 1]^T \cdot [-0.5, 0.5])/2$$

$$d_w = (-1 + 0.5)/2 = -0.25$$

Com GAAR:

1. Peso por classe:

$$c = np.where(y == 1, self.c_{minority}, 1.0) = [10.0, 1.0]$$

2. Derivada do sigmoide:

$$sigmoid\_deriv = probabilities * (1 - probabilities) = [0.25, 0.25]$$

3. Vetor `scale` :

$$x\_norms\_sq = np.sum(X ** 2, axis = 1) = [4.0, 1.0]$$

$$scale = 2 * c * error * sigmoid\_deriv * x\_norms\_sq = [-10, 0.25]$$

4. Ajuste do gradiente:

$$d_{w-gaar} = 0.1/2 * X^T * scale = 0.05 * (2 * (-10) + 1 * 0.25) = -0.9875$$

5. Gradiente final:

$$dw = dw_{default} + d_{w-gaar} = -1.2375$$

# Add-ons - Class Asymmetric Gradient Descent

— Proposta semelhante ao GAAR, mas não utiliza o erro, e sim a curvatura de uma amostra:

$$v_i = p_i(1 - p_i)$$

— Utilizando a Hessiana empírica:

$$H = \frac{1}{N} \sum_{i=1}^N v_i x_i x_i^T$$

O CAGD separa esta curvatura por classe:

$$H_0 = \frac{1}{N} \sum_{i \in \{j | y_j=0\}} v_i x_i x_i^T$$

$$H_1 = \frac{1}{N} \sum_{i \in \{j | y_j=1\}} v_i x_i x_i^T$$

Depois constrói o preconditionador:

$$P = H_0 + \alpha H_1 + \epsilon I$$

E aplica esse preconditionador ao gradiente batch  $dw$  :

$$dw_{CAGD} = P^{-1} dw$$

# Add-ons - Class Asymmetric Gradient Descent

— Proposta semelhante ao GAAR, mas não utiliza o erro, e sim a curvatura de uma amostra:

$$v_i = p_i(1 - p_i)$$

— Utilizando a Hessiana empírica:

$$H = \frac{1}{N} \sum_{i=1}^N v_i x_i x_i^T$$

O CAGD separa esta curvatura por classe:

$$H_0 = \frac{1}{N} \sum_{i \in \{j | y_j=0\}} v_i x_i x_i^T$$

$$H_1 = \frac{1}{N} \sum_{i \in \{j | y_j=1\}} v_i x_i x_i^T$$

Depois constrói o preconditionador:

$$P = H_0 + \alpha H_1 + \epsilon I$$

E aplica esse preconditionador ao gradiente batch  $dw$ :

$$dw_{CAGD} = P^{-1} dw$$

— Exemplo do exercício anterior ( $\alpha=5.0$ ):

| X | y | erro | pesos iniciais (w) |
|---|---|------|--------------------|
| 2 | 1 | -0,5 | 0                  |
| 1 | 0 | 0,5  | 0                  |

Temos:

$$H_0 = \frac{1}{2}(0.5(1 - 0.5))1^2 = 0.125$$

$$H_1 = \frac{1}{2}(0.5(1 - 0.5))2^2 = 0.5$$

$$P = 0.125 + 5 * 0.5 = 2.625$$

Então:

$$dw_{CAGD} = P^{-1} dw = \frac{-0.25}{2.625} = -0.0952$$

# Add-ons - F1 Dynamic Threshold

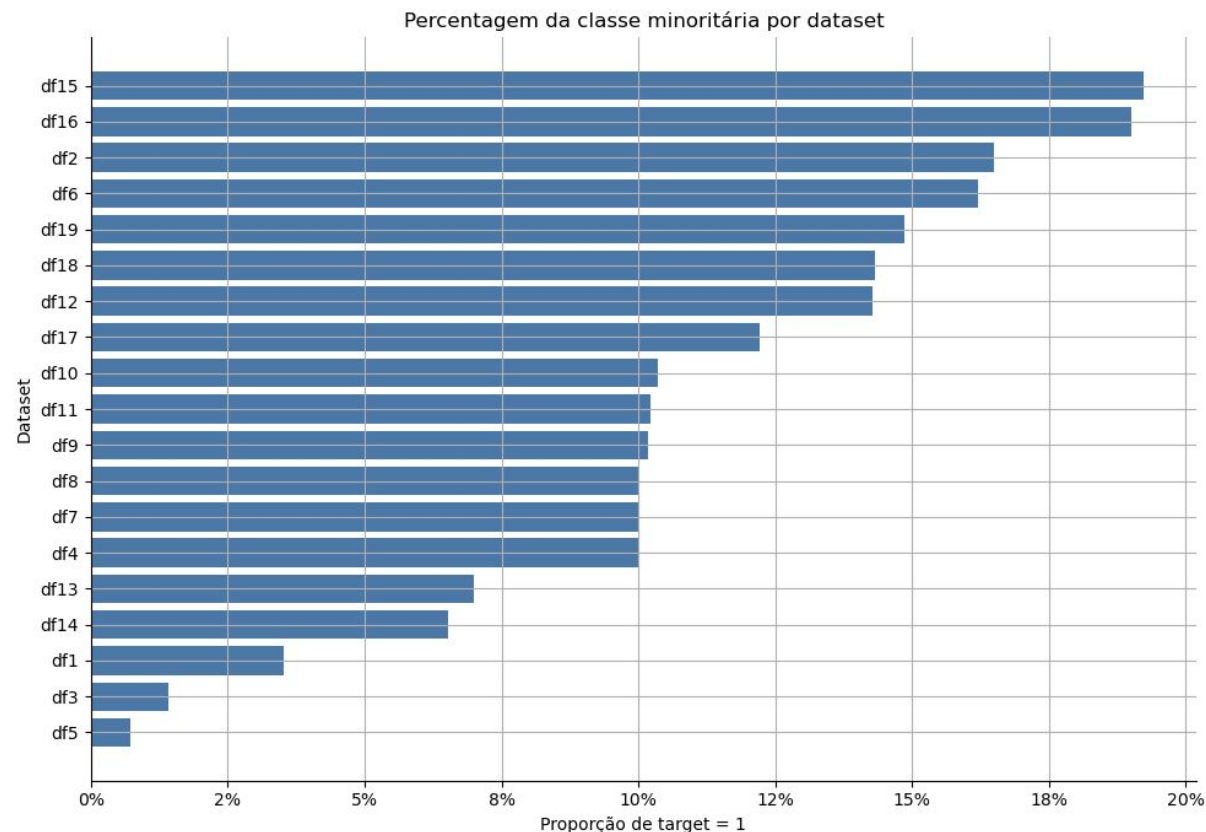
- Não altera o treino, e sim o threshold após o treino
- Utiliza um dataset de validação, para os meus testes peguei aleatoriamente 20% do dataset de treino (o que não é o ideal)
- Para cada threshold entre 0.01 e 0.99, calcula-se o F1 da classe positiva (minoritária)

$$F1 = \frac{2TP}{2TP+FP+FN}$$

- Threshold escolhido minimiza F1
- F1 “maximiza” ao mesmo tempo a precision e a recall
  - Precision: dos minoritários marcados, quantos eram realmente minoritários
  - Recall: dos minoritários, quantos acertos

# Análise dos add-ons

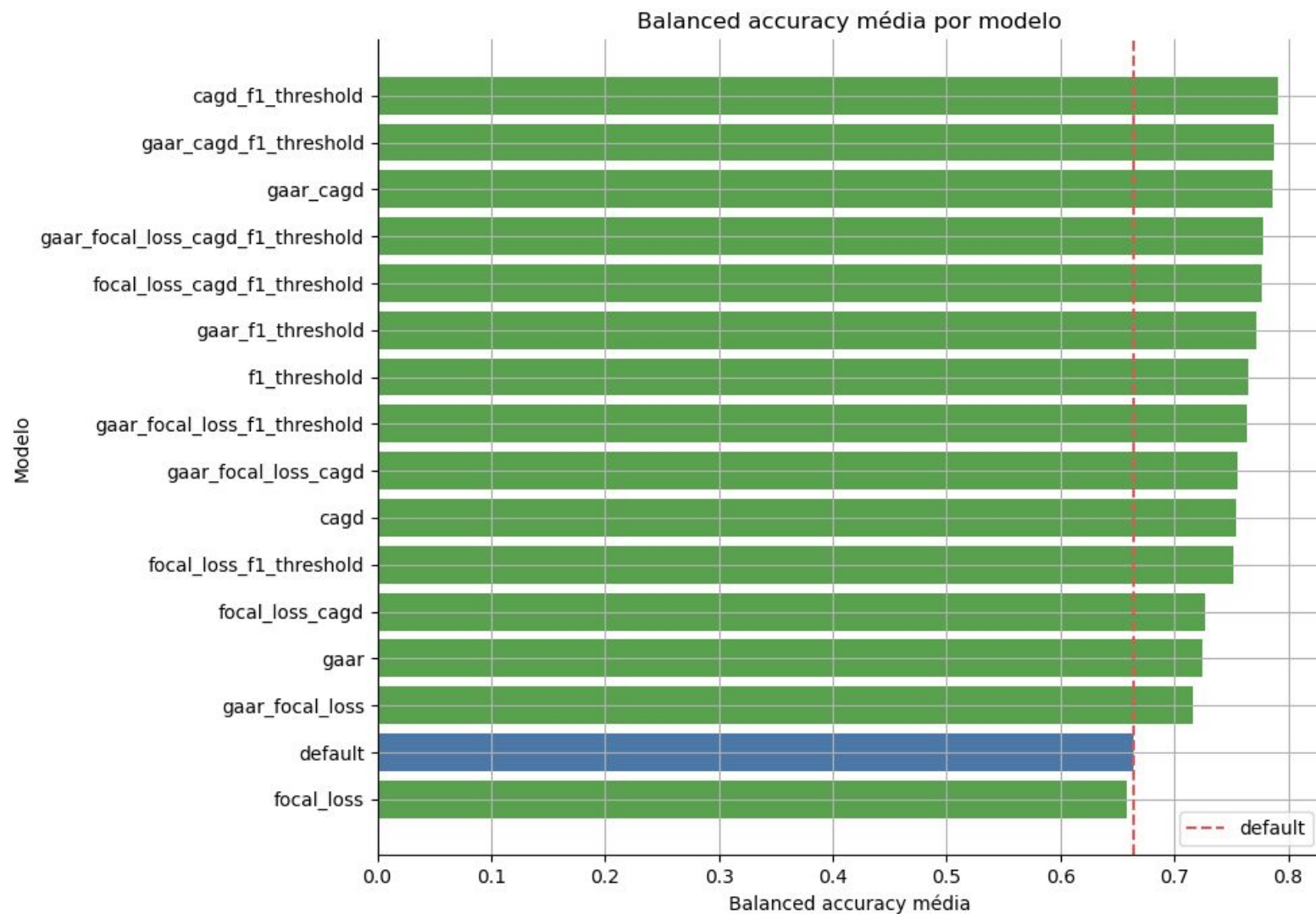
- Carregados e tratados 19 datasets ([datasets.py](#)).
- Classe minoritária positiva
- 16 ( $2^4$ ) modelos diferentes
- 20 seeds
- Total de 6080 execuções -> uma execução corresponde ao treinamento de um dataset, em uma modelo específico (ou combinação de modelos)
- Para rodar utilizar o arquivo (`src/analyze_addon_performance.py`)
- Resultados em `/outputs`



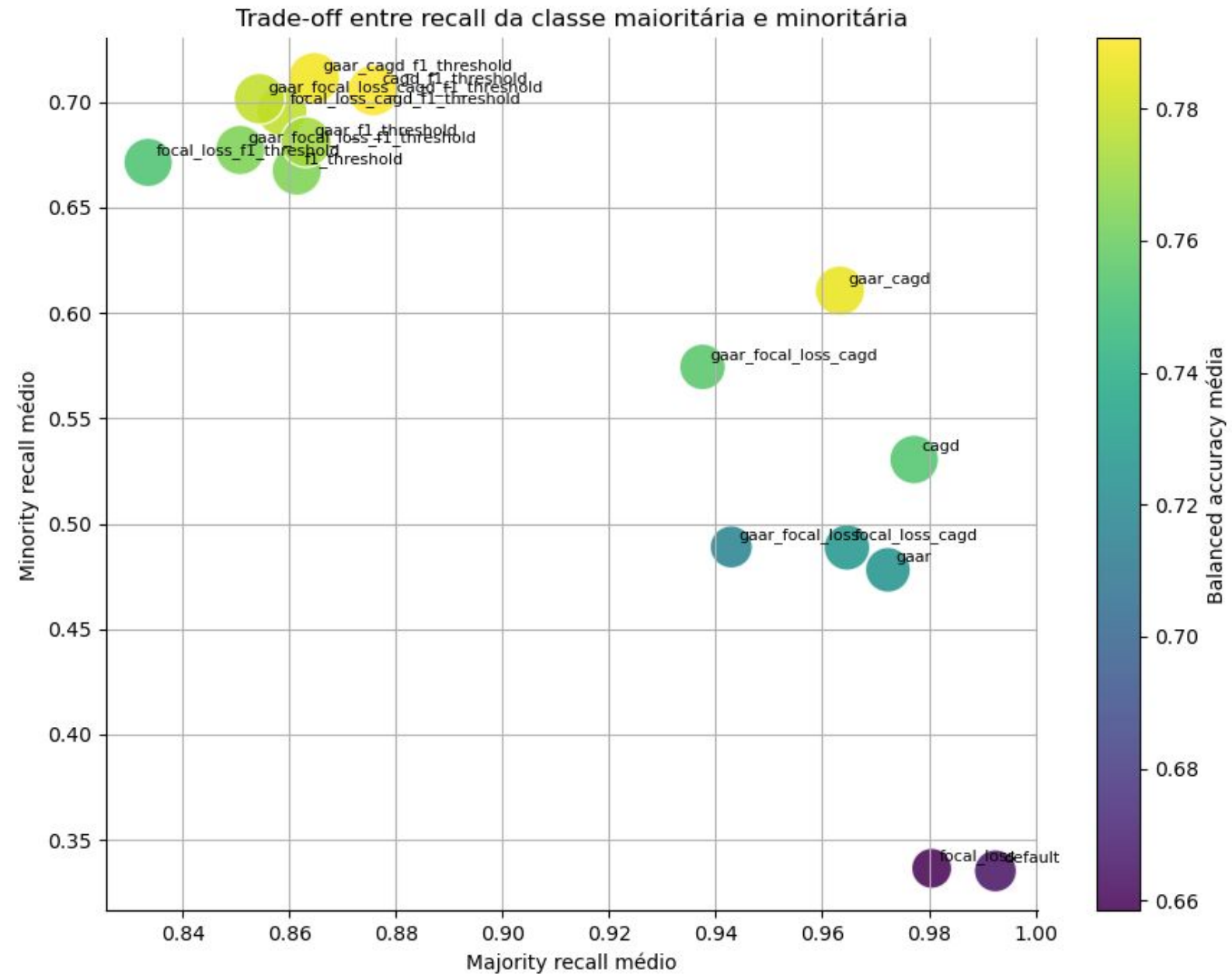
# Resultados

|    | model                             | balanced_accuracy_mean | minority_recall_mean | minority_f1_mean | average_precision_mean | roc_auc_mean |
|----|-----------------------------------|------------------------|----------------------|------------------|------------------------|--------------|
| 0  | cagd_f1_threshold                 | 0.7907                 | 0.7056               | 0.6048           | 0.6793                 | 0.8933       |
| 1  | gaar_cagd_f1_threshold            | 0.7881                 | 0.7115               | 0.6069           | 0.6824                 | 0.8860       |
| 2  | gaar_cagd                         | 0.7869                 | 0.6106               | 0.5614           | 0.6824                 | 0.8860       |
| 3  | gaar_focal_loss_cagd_f1_threshold | 0.7780                 | 0.7016               | 0.5962           | 0.6679                 | 0.8784       |
| 4  | focal_loss_cagd_f1_threshold      | 0.7773                 | 0.6959               | 0.5852           | 0.6558                 | 0.8815       |
| 5  | gaar_f1_threshold                 | 0.7721                 | 0.6811               | 0.5820           | 0.6569                 | 0.8727       |
| 6  | f1_threshold                      | 0.7645                 | 0.6676               | 0.5622           | 0.6325                 | 0.8649       |
| 7  | gaar_focal_loss_f1_threshold      | 0.7640                 | 0.6773               | 0.5614           | 0.6353                 | 0.8648       |
| 8  | gaar_focal_loss_cagd              | 0.7560                 | 0.5744               | 0.4639           | 0.6679                 | 0.8784       |
| 9  | cagd                              | 0.7538                 | 0.5304               | 0.5406           | 0.6793                 | 0.8933       |
| 10 | focal_loss_f1_threshold           | 0.7525                 | 0.6714               | 0.5328           | 0.5969                 | 0.8501       |
| 11 | focal_loss_cagd                   | 0.7267                 | 0.4887               | 0.4617           | 0.6558                 | 0.8815       |
| 12 | gaar                              | 0.7252                 | 0.4781               | 0.4395           | 0.6569                 | 0.8727       |
| 13 | gaar_focal_loss                   | 0.7159                 | 0.4889               | 0.3760           | 0.6353                 | 0.8648       |
| 14 | default                           | 0.6639                 | 0.3352               | 0.3722           | 0.6325                 | 0.8649       |
| 15 | focal_loss                        | 0.6585                 | 0.3365               | 0.3244           | 0.5969                 | 0.8501       |

# Resultados

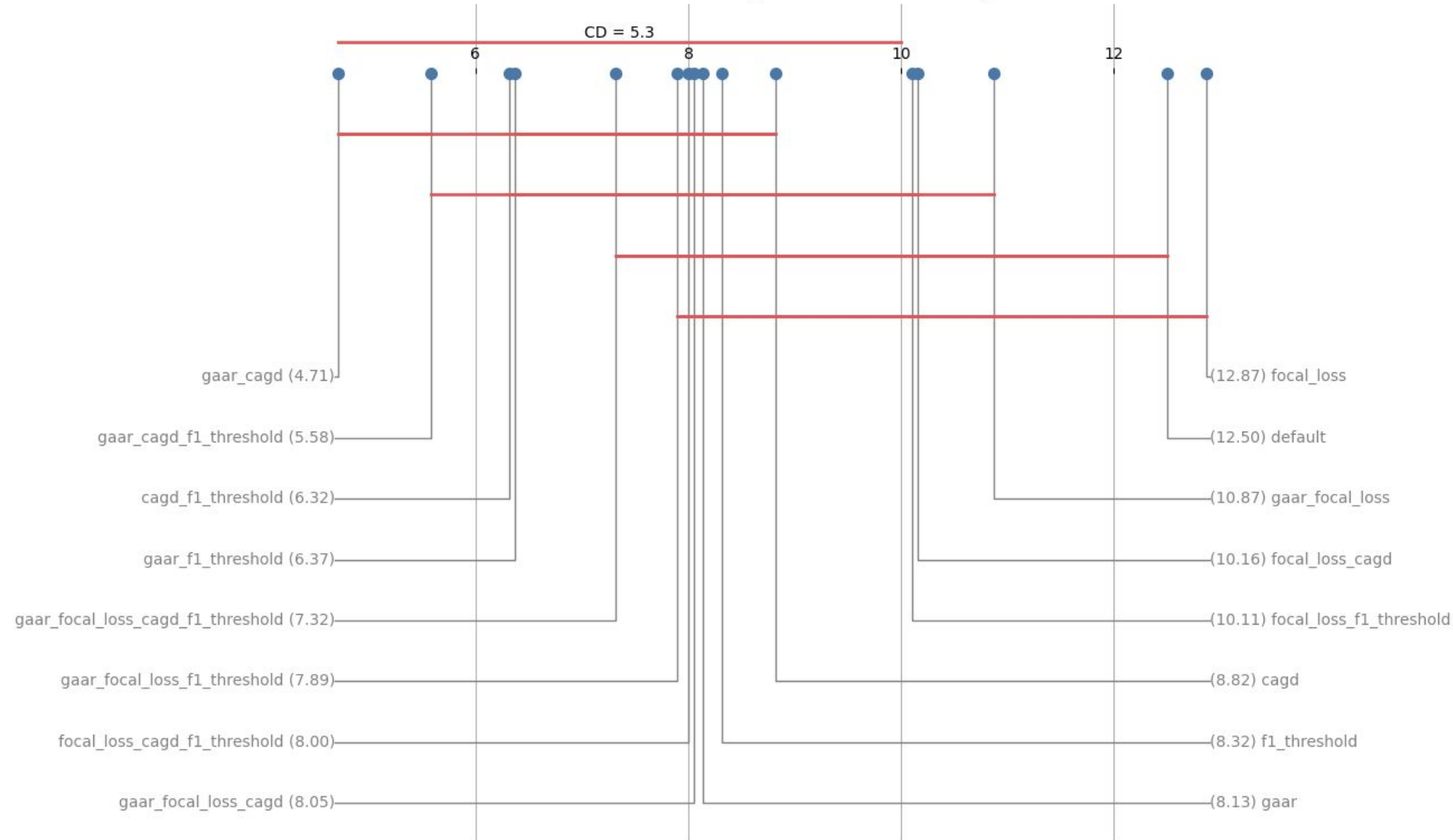


# Resultados



# Resultados

Critical Difference diagram - balanced accuracy



# Conclusões

|   | Critério                          | Escolha recomendada |
|---|-----------------------------------|---------------------|
| 0 | Maximizar balanced accuracy média | cagd_f1_threshold   |
| 1 | Maior consistência por rank       | gaar_cagd           |
| 2 | Solução simples e rápida          | f1_threshold        |
| 3 | Não usar isoladamente neste setup | focal_loss          |

- CAGD foi o add-on de treino mais consistente
- F1Threshold aumentou recall e F1 da minoria, com custos em falsos positivos
- CAGD + F1 Threshold teve melhor balanced accuracy média.
- GAAR + CAGD foi mais equilibrado por rank médio.
- Possível trabalhos futuros:
  - Os parâmetros foram tratados como constantes para reduzir a complexidade, mas eles podem ser dinamizados com base nos datasets
  - Treinar com um maior número de datasets
  - Há um data leakage na preparação de datasets que pode ser facilmente corrigida

# Reprodutibilidade

- Arquivos principais:
- `'src/logistic_regression.py'`: Implementação da LR
- `'src/analyze_addon_performance.py'`: Roda os 6080 exemplos
- `'src/datasets.py'`: ETL dos datasets a partir de `/class_imbalance`
- **`analyze_addon_performance_results.ipynb`: notebook principal**
- Saídas principais:
- `'outputs/addon_performance/raw_results.csv'`
- `'outputs/addon_performance/model_summary.csv'`
- `'outputs/addon_performance/dataset_summary.csv'`
- `'outputs/addon_performance/config.json'`
- Reprodutibilidade (**`analyze_addon_performance_results.ipynb`**):
- `'RUN_FULL_EXPERIMENT = False'`: ler resultados salvos
- `'RUN_FULL_EXPERIMENT = True'`: recalcular a experiência

